Lecture 2

Tokenization for Language Modeling

Lecture Notes

MAI554 – Deep Learning for NLP

Prof. Anis Koubaa

College of Engineering, Alfaisal University

akoubaa@alfaisal.edu

Spring 2025

How to Use These Notes

These notes distill the key ideas from Lecture 2 into a logical flow. Each section follows a pattern: **intuition first, then mechanics, then a concrete example**. Self-check questions appear throughout the text. Full solutions are collected in the Appendix.

Contents

1 The Language Modeling Pipeline **2**
- 1.1 From Text to Prediction . . . 2
- 1.2 Worked Example . . . 2

2 From Vocabulary to Token Indices **2**
- 2.1 The Vocabulary as a Lookup Table . . . 2
- 2.2 The Three-Step Process . . . 3
- 2.3 Worked Example . . . 3

3 Handling Unknown Words: The OOV Problem **3**
- 3.1 The Problem . . . 3
- 3.2 Solution 1: The `<UNK>` Token . . . 3
- 3.3 Solution 2: Subword Tokenization . . . 4

4 What is Tokenization and Why It Matters **4**
- 4.1 Definition . . . 4
- 4.2 Three Levels of Tokenization . . . 4
- 4.3 Why Tokenization Matters for Performance . . . 4

5 Comparing Tokenizers: GPT-2 vs GPT-4 **5**
- 5.1 Key Differences . . . 5
- 5.2 Why a Larger Vocabulary Helps . . . 5

6 Unicode and Why It's Not Enough **6**
- 6.1 What is Unicode? . . . 6
- 6.2 Encoding Schemes: UTF-8, UTF-16, UTF-32 . . . 6
- 6.3 Why Not Use Unicode Code Points as Tokens? . . . 6
- 6.4 Why Not Use Raw UTF-8 Bytes? . . . 6

7 Character-Level Tokenization **7**
- 7.1 Definition and Process . . . 7
- 7.2 Worked Example: Shakespeare . . . 7
- 7.3 Advantages . . . 7
- 7.4 Disadvantages . . . 7

8 Word-Level Tokenization **8**
- 8.1 Definition and Process . . . 8
- 8.2 Worked Example: Shakespeare . . . 8
- 8.3 Advantages . . . 8
- 8.4 Disadvantages . . . 8

9 Subword Tokenization: The Best of Both Worlds **9**
9.1 Why Subword Tokenization? 9
9.2 How It Works (Intuition) 9
9.3 Arabic: A Case for Morphological Awareness 9

10 Byte Pair Encoding (BPE) **10**
10.1 The Algorithm 10
10.2 Worked Example: "low lower lowest" 10
10.3 BPE in GPT-2: Byte-Level BPE 11

11 TikToken: OpenAI's Tokenization Library **12**
11.1 Basic Usage 12
11.2 Comparing Encodings 12

12 Summary: Choosing the Right Tokenization **12**
12.1 Comparison Table 13
12.2 The Evolution 13

Appendix: Solutions to Self-Check Questions **14**

The Language Modeling Pipeline

Before diving into tokenization, let us see where it fits in the **big picture** of language modeling.

From Text to Prediction

A language model takes raw text as input and predicts the next token. The pipeline has five stages:

InputText ➔ Tokenization ➔ Token +PositionalEmbeddings ➔ TransformerBlock ➔ Next TokenPrediction

- **Input Text:** Raw string, e.g. `"the mouse like eating"`
- **Tokenization:** Splits text into tokens and maps each to an integer index
- **Embeddings:** Each token index is mapped to a dense vector (token embedding), and a positional embedding is added to encode word order
- **Transformer Block:** Processes all token embeddings in parallel using self-attention
- **Next Token Prediction:** Outputs a probability distribution over the vocabulary

Worked Example

Consider the input: `"the mouse like eating"`

Stage	Operation	Result
1	Tokenize	`["the", "mouse", "like", "eating"]`
2	Map to indices	`[1, 5, 12, 8]`
3	Embed + position	4 dense vectors, each encoding meaning + position
4	Transformer	Contextualized representations
5	Predict next	"cheese" with 85% probability

Key Insight

Tokenization is the **first and most critical** step in the pipeline. If the tokenizer breaks words poorly, the model receives garbage input—no amount of sophisticated architecture can recover from bad tokenization. **Garbage in, garbage out.**

From Vocabulary to Token Indices

The Vocabulary as a Lookup Table

A **vocabulary** is a fixed set of known tokens, each assigned a unique integer index. Think of it as a dictionary that maps strings to numbers:

Token	Index	Token	Index	Token	Index	Token	Index
the	0	jumping	1	fox	2	is	3
brown	4	.	5	!	6	quick	7

The Three-Step Process

1. **Splitting:** Break the input text into individual tokens
2. **Lookup:** Find each token in the vocabulary
3. **Indexing:** Replace each token with its integer index

Worked Example

Input: `"The quick brown fox is jumping."`

Step	Operation	Result
1	Split into tokens	`["The", "quick", "brown", "fox", "is", "jumping", "."]`
2	Lookup in vocabulary	Each token found (assuming case-insensitive match)
3	Map to indices	`[0, 7, 4, 2, 3, 1, 5]`

The resulting integer sequence `[0, 7, 4, 2, 3, 1, 5]` is what the model actually receives. Each number is then used to look up a dense vector in the embedding matrix.

Important

The vocabulary is **fixed** at training time. Any word not in the vocabulary cannot be represented directly. This leads to the **out-of-vocabulary (OOV) problem**, discussed next.

Handling Unknown Words: The OOV Problem

The Problem

Real-world text is messy. Users type misspellings, slang, technical terms, and words from many languages. If a word is not in the vocabulary, the tokenizer cannot assign it an index. This is the **Out-of-Vocabulary (OOV)** problem.

Example: Suppose our vocabulary is: `{hello, how, are, you, ?, !}`
Input: `"hello universe!"`
The word `"universe"` is **not** in the vocabulary. What do we do?

Solution 1: The `<UNK>` Token

The simplest approach: add a special `<UNK>` (unknown) token to the vocabulary. Any word not found is mapped to `<UNK>`:

`"hello universe!"` → `["hello", "<UNK>", "!"]`

Problem: All unknown words collapse into the same representation. The model cannot distinguish between "universe", "galaxy", or "xyzzy"—they all become `<UNK>`.

Solution 2: Subword Tokenization

Instead of treating entire words as tokens, break unknown words into **smaller pieces** that *are* in the vocabulary:

`"universe"` → `["uni", "verse"]`

Both `"uni"` and `"verse"` are common substrings likely found in any reasonable vocabulary. The model can still extract meaning from these pieces. This is the idea behind **subword tokenization**, which we will explore in depth later.

Key Insight

Subword tokenization eliminates the OOV problem entirely. Any string—no matter how unusual—can be decomposed into known subword units, down to individual characters or bytes if necessary. This is why modern LLMs (GPT, BERT, LLaMA) all use subword tokenization.

Self-Check Q1

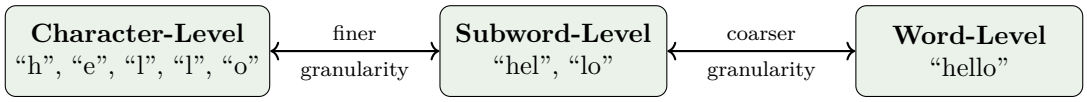
Why is mapping all unknown words to a single `<UNK>` token problematic for a language model? Give a concrete example where this causes the model to fail. *(Solution in Appendix)*

What is Tokenization and Why It Matters

Definition

Tokenization is the process of converting raw text into smaller units called **tokens**. A token can be a word, a subword, or a character—the choice of granularity has major consequences for model performance.

Three Levels of Tokenization



- **Character-level:** Each character is a token. Tiny vocabulary (∼50–100), but very long sequences.
- **Subword-level:** Words are split into meaningful pieces. Balanced vocabulary (∼30K–100K) and moderate sequence length. **This is what modern LLMs use.**
- **Word-level:** Each word is a token. Semantically rich, but vocabulary can be huge and OOV is inevitable.

Why Tokenization Matters for Performance

The choice of tokenizer directly affects model quality in several ways:

1. **Spelling errors:** Poor tokenizers break misspelled words into meaningless fragments
2. **Arithmetic:** How numbers are tokenized affects whether the model can "see" individual digits (e.g., "380" as one token vs. "3", "8", "0")

3. **Non-English languages:** Many tokenizers are trained primarily on English; other languages get split into excessively many tokens, reducing quality and increasing cost
4. **Inconsistent handling:** The same concept may be tokenized differently depending on context (e.g., leading whitespace, capitalization)

The Golden Rule

"The better the tokenization, the smarter the model." — A well-designed tokenizer gives the model cleaner, more consistent input, allowing it to focus on learning language patterns rather than fighting tokenization artifacts.

Comparing Tokenizers: GPT-2 vs GPT-4

OpenAI significantly improved their tokenizer between GPT-2 and GPT-4. Understanding these differences illustrates why tokenizer design matters.

Key Differences

Dimension	GPT-2	GPT-4
Vocabulary size	∼50,257	∼100,256
Encoding name	`r50k_base`	`cl100k_base`
Token efficiency	Lower	Higher (fewer tokens for same text)
Whitespace handling	Inconsistent	Improved
Non-English support	Poor (many tokens per word)	Better (fewer tokens per word)
Number handling	Splits inconsistently	More consistent digit handling
Special characters	Limited	Better coverage

Why a Larger Vocabulary Helps

A larger vocabulary means:

- **Fewer tokens per text** — common words and phrases become single tokens
- **Shorter sequences** — reduces computation ($O(n^2)$ attention cost)
- **Better multilingual support** — room for non-English subwords
- **Lower API cost** — users pay per token, so efficiency matters

Trade-off: A larger vocabulary means a larger embedding matrix, which increases model size and memory usage. The sweet spot for modern LLMs is typically 30K–100K tokens.

Self-Check Q2

GPT-4's tokenizer uses ∼100K tokens while GPT-2 uses ∼50K. Why not simply use a vocabulary of 1 million tokens? What would be the disadvantages? *(Solution in Appendix)*

Unicode and Why It's Not Enough

What is Unicode?

Before tokenization, text is represented as a sequence of **Unicode code points**. Unicode is a universal standard that assigns a unique number to every character in every language:

Character	Code Point	Description
A	U+0041 (65)	Latin capital letter A
a	U+0061 (97)	Latin lowercase letter a
#	U+0023 (35)	Number sign
[smiley face]	U+1F60A (128522)	Smiling face emoji

Encoding Schemes: UTF-8, UTF-16, UTF-32

Unicode code points must be encoded as bytes for storage and transmission:

Encoding	Bytes/char	Advantage	Disadvantage
UTF-8	1–4 (variable)	Compact for ASCII/English	Variable-length complicates indexing
UTF-16	2 or 4	Good for Asian scripts	Wastes space for ASCII
UTF-32	4 (fixed)	Simple indexing	Wastes space (4× ASCII)

Why Not Use Unicode Code Points as Tokens?

One might ask: why not simply use each Unicode code point as a token? There are several problems:

1. **Huge vocabulary:** Unicode defines over **150,000** characters. The embedding matrix would be enormous.
2. **Dynamic standard:** New characters (especially emojis) are added regularly. The vocabulary would need constant updating.
3. **Long sequences:** Each character becomes a separate token, leading to very long sequences.
4. **Quadratic attention cost:** Transformer self-attention is $O(n^2)$ in sequence length—longer sequences are dramatically more expensive.

Why Not Use Raw UTF-8 Bytes?

Another idea: use the 256 possible byte values as the vocabulary. This gives a tiny, fixed vocabulary, but:

1. **Too few tokens:** Only 256 unique tokens—the model has almost no "vocabulary" to work with
2. **Very long sequences:** A single emoji (e.g., 4 bytes in UTF-8) becomes 4 tokens; words in non-Latin scripts become 2–4 tokens per character

3. **No semantic structure:** Individual bytes carry no linguistic meaning

The Dilemma

Unicode code points: vocabulary too **large** (∼150K). Raw UTF-8 bytes: vocabulary too **small** (256). We need something in between—a vocabulary of 30K–100K meaningful **subword** tokens. This is exactly what BPE and other subword algorithms provide.

Self-Check Q3

If we used raw UTF-8 bytes as our vocabulary (256 tokens), how many tokens would the English word "hello" require? How about a Chinese character like " " (which uses 3 bytes in UTF-8)? What does this tell us about sequence length? *(Solution in Appendix)*

Character-Level Tokenization

Definition and Process

The simplest tokenization strategy: split text into **individual characters**, including spaces and punctuation. Each unique character becomes a token in the vocabulary.

Worked Example: Shakespeare

Input: `"To be, or not to be"`

```
["T", "o", " ", "b", "e", ",", " ", "o", "r", " ", "n", "o", "t", " ", "t",
                                "o", " ", "b", "e"]
```

This produces **19 tokens** from 19 characters (spaces included). In Shakespeare's complete works (∼5 million characters), the vocabulary contains only about **65 unique characters** (letters, digits, punctuation, whitespace).

Advantages

- **Compact vocabulary:** Only ∼50–100 unique tokens (for English). The embedding matrix is tiny.
- **Language-independent:** Works for any language without language-specific preprocessing.
- **No OOV problem:** Any text can be represented—there are no unknown characters.
- **Captures subword patterns:** The model can learn morphological patterns (e.g., "-ing", "-ed", "un-") from raw characters.

Disadvantages

- **Very long sequences:** A 500-word paragraph might produce 3,000+ character tokens.
- **Quadratic attention cost:** Self-attention is $O(n^2)$. Doubling sequence length quadruples the cost.
- **No semantic structure:** Individual characters carry no meaning. The model must learn that "c", "a", "t" together mean a feline—a much harder learning task.
- **Slower training:** More tokens per example means more computation per training step.

When to Use Character-Level Tokenization

Character-level models are best for tasks where **character patterns matter**: spelling correction, handwriting recognition, transliteration, or processing languages with very large character sets (e.g., Chinese). They are generally **not used** for large-scale language models due to sequence length constraints.

Word-Level Tokenization

Definition and Process

Split text on whitespace and punctuation boundaries. Each unique word becomes a token in the vocabulary.

Worked Example: Shakespeare

Input: `"To be, or not to be"`

```
["To", "be", ",", "or", "not", "to", "be"]
```

This produces **7 tokens**—much fewer than the 19 character tokens. However, Shakespeare's complete works contain approximately **30,000 unique words**—a much larger vocabulary.

Advantages

- **Captures word semantics:** Each token is a meaningful linguistic unit. The model can directly learn word meanings.
- **Shorter sequences:** Far fewer tokens per text, enabling longer context windows.
- **Intuitive:** Aligns with how humans think about language.

Disadvantages

- **Large vocabulary:** English alone has 170,000+ words; technical domains add more. The embedding matrix becomes enormous.
- **OOV problem:** New words, misspellings, and rare terms cannot be represented.
- **Morphological blindness:** The model treats "run", "running", "runner", and "runs" as *completely unrelated* tokens, wasting capacity learning the same root meaning four times.
- **Language-dependent:** Whitespace-based splitting does not work for Chinese, Japanese, Thai, and other languages without word boundaries.

The Core Trade-off

Character-level tokenization: small vocabulary, long sequences, no semantics.
Word-level tokenization: large vocabulary, short sequences, rich semantics but OOV issues.

Can we get the best of both worlds? Yes—with **subword tokenization**.

Self-Check Q4

Consider the words "run", "running", "runner", and "runs". Explain why word-level tokenization wastes model capacity for these words, and how subword tokenization would handle them more efficiently. *(Solution in Appendix)*

Subword Tokenization: The Best of Both Worlds

Why Subword Tokenization?

We have seen two extremes:

- **Characters:** Too fine-grained—sequences too long, no semantics.
- **Words:** Too coarse—vocabulary too large, OOV problem.

Subword tokenization **bridges the gap**. The goals are:

1. **Optimize vocabulary size:** Keep it manageable (30K–100K tokens)
2. **Eliminate OOV:** Any word can be decomposed into known subwords
3. **Represent morphemes:** Capture meaningful sub-units like prefixes, suffixes, and roots

How It Works (Intuition)

Common words ("the", "is", "and") are kept as single tokens. Rare or complex words are split into smaller pieces:

Word	Subword Tokens
`unhappiness`	`["un", "happiness"]` or `["un", "happ", "iness"]`
`tokenization`	`["token", "ization"]`
`transformer`	`["transform", "er"]`
`xylophone`	`["x", "yl", "ophone"]`

Arabic: A Case for Morphological Awareness

Arabic is a **morphologically rich** language built on a root-and-pattern system. The triliteral root **k-t-b** (related to writing) generates many words:

Arabic Word	Transliteration	Meaning
kataba	kataba	he wrote
kaatib	kaatib	writer
kitaab	kitaab	book
kutub	kutub	books
katabtu	katabtu	I wrote
maktaba	maktaba	library

A good subword tokenizer can learn to recognize the shared root `ktb` across all these forms, enabling the model to transfer knowledge between related words. Word-level tokenization would treat each form as an independent, unrelated token.

Key Insight

Subword tokenization is not just an engineering trick—it captures **linguistic structure**. Morphemes (the smallest meaningful units) are naturally discovered by the algorithm, enabling the model to generalize across word forms.

Byte Pair Encoding (BPE)

The Algorithm

BPE is the most widely used subword tokenization algorithm. It was originally a data compression technique (Gage, 1994) and was adapted for NLP by Sennrich et al. (2016). The algorithm is elegant in its simplicity:

BPE Algorithm

1. Start with a vocabulary of all individual characters (or bytes)
2. Count all consecutive pairs of tokens in the training corpus
3. Find the most frequent pair
4. Merge that pair into a new single token; add it to the vocabulary
5. Repeat steps 2–4 until the desired vocabulary size is reached

Worked Example: "low lower lowest"

Let us trace through BPE step by step.

Initial state: Represent each word as a sequence of characters (using byte values for clarity):

```
"low"    -> [108, 111, 119]            # l, o, w
"lower"  -> [108, 111, 119, 101, 114] # l, o, w, e, r
"lowest" -> [108, 111, 119, 101, 115, 116] # l, o, w, e, s, t
```

Initial vocabulary: all unique bytes appearing in the corpus.

Merge 1 — Most frequent pair: (108, 111) = "lo"

The pair `(108, 111)` (i.e., "l" followed by "o") appears 3 times (once in each word). This is the most frequent pair.
Action: Create a new token 256 = "lo". Replace all occurrences:

```
"low"    -> [256, 119]            # lo, w
"lower"  -> [256, 119, 101, 114] # lo, w, e, r
"lowest" -> [256, 119, 101, 115, 116] # lo, w, e, s, t
```

Vocabulary size: original bytes + 1 new token.

Merge 2 — Most frequent pair: (256, 119) = "low"

The pair `(256, 119)` (i.e., "lo" followed by "w") appears 3 times.
Action: Create a new token 257 = "low". Replace all occurrences:

```
"low"    -> [257]                 # low
"lower"  -> [257, 101, 114]      # low, e, r
"lowest" -> [257, 101, 115, 116] # low, e, s, t
```

Merge 3 — Most frequent pair: (257, 101) = "lowe"

The pair `(257, 101)` (i.e., "low" followed by "e") appears 2 times (in "lower" and "lowest").
Action: Create a new token 258 = "lowe". Replace all occurrences:

```
"low"    -> [257]              # low
"lower"  -> [258, 114]        # lowe, r
"lowest" -> [258, 115, 116]   # lowe, s, t
```

The process continues until the target vocabulary size is reached. Notice how BPE naturally discovers meaningful subword units: "low" becomes a single token, and "lower" and "lowest" share the common prefix "lowe".

BPE in GPT-2: Byte-Level BPE

GPT-2 uses a variant called **byte-level BPE**:

1. **Base vocabulary:** 256 raw UTF-8 byte values (not Unicode characters). This ensures *any* text can be tokenized—no `<UNK>` tokens needed.
2. **Regex pre-splitting:** Before applying BPE merges, the text is split using a regex pattern that separates:
   - Words (with optional leading space)
   - Numbers
   - Punctuation
   - Whitespace sequences
3. **No cross-category merges:** BPE merges are only applied *within* each regex-matched chunk. This prevents nonsensical merges like merging a letter with a digit or merging across word boundaries.

```
# GPT-2 regex pattern (simplified)
import regex
pattern = r"""'s|'t|'re|'ve|'m|'ll|'d
|␣?\p{L}+
|␣?\p{N}+
|␣?[^\s\p{L}\p{N}]+
|␣\s+"""
```

Why Byte-Level?

By starting from bytes instead of Unicode characters, GPT-2 can tokenize **any text in any language or encoding** without ever producing an unknown token. The 256-byte base vocabulary is universal, compact, and fixed. BPE merges then build up from bytes to meaningful subword units.

Self-Check Q5

In BPE, why do we merge the most *frequent* pair at each step? What would happen if we instead merged the *least* frequent pair? *(Solution in Appendix)*

TikToken: OpenAI's Tokenization Library

TikToken is OpenAI's official, open-source tokenization library used in GPT-2, GPT-3, GPT-3.5, and GPT-4. It provides fast, production-grade BPE tokenization.

Basic Usage

```
import tiktoken

# Load GPT-4's tokenizer
enc = tiktoken.encoding_for_model("gpt-4")

# Encode text to token IDs
tokens = enc.encode("Hello, world!")
print(tokens)    # e.g., [9906, 11, 1917, 0]

# Decode token IDs back to text
text = enc.decode(tokens)
print(text)      # "Hello, world!"

# Count tokens (useful for API cost estimation)
num_tokens = len(enc.encode("Your text here"))
```

Comparing Encodings

Encoding	Used by	Vocab Size
`r50k_base`	GPT-2	50,257
`p50k_base`	GPT-3 (Codex)	50,281
`cl100k_base`	GPT-3.5, GPT-4	100,256

Practical Tip

When using OpenAI's API, you **pay per token**. Use TikToken locally to estimate costs *before* making API calls. Different models use different tokenizers, so always match the encoding to the model you plan to use.

Self-Check Q6

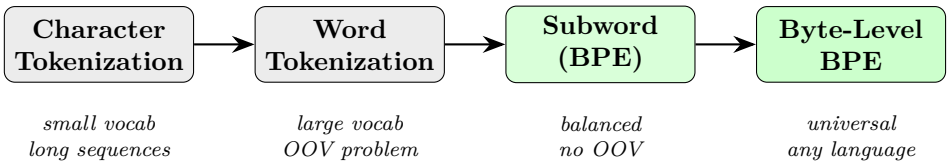
Using TikToken, the sentence "I love deep learning" is encoded as 4 tokens by GPT-4's tokenizer. If the same sentence is encoded by GPT-2's tokenizer, would you expect more tokens, fewer tokens, or the same number? Explain your reasoning. *(Solution in Appendix)*

Summary: Choosing the Right Tokenization

Comparison Table

Dimension	Character-Level	Word-Level	Subword (BPE)
Vocabulary size	Very small (∼50–100)	Very large (∼100K+)	Moderate (∼30K–100K)
Sequence length	Very long	Short	Moderate
OOV handling	No OOV	Frequent OOV	No OOV
Semantic content	None per token	Rich per token	Moderate per token
Morphology	Must learn from scratch	Ignores it	Captures naturally
Computation cost	High ($O(n^2)$ with long n)	Low	Moderate
Used in practice	Specialized tasks	Legacy systems	**All modern LLMs**

The Evolution



Bottom Line

Modern language models universally use **subword tokenization** (specifically byte-level BPE or similar algorithms like SentencePiece). This gives the optimal trade-off between vocabulary size, sequence length, and linguistic coverage. If you are building or fine-tuning an LLM, the tokenizer is not an afterthought—it is a **design decision** that directly impacts model quality.

Self-Check Q7

You are designing a tokenizer for a multilingual model that must support English, Arabic, Chinese, and code. Would you choose character-level, word-level, or subword tokenization? Justify your choice by addressing each language's specific challenges. *(Solution in Appendix)*

Appendix: Solutions to Self-Check Questions

Q1: Why is mapping all unknown words to <UNK> problematic?

When all unknown words map to the same <UNK> token, the model **loses all ability to distinguish** between them. For example, consider a medical NLP system with a limited vocabulary:

- "The patient has **pneumonia**" → ["The", "patient", "has", "<UNK>"]
- "The patient has **diabetes**" → ["The", "patient", "has", "<UNK>"]

Both sentences produce *identical* token sequences. The model cannot distinguish between pneumonia and diabetes—a potentially dangerous failure. Subword tokenization would split these into recognizable pieces ("pneum", "onia" and "diab", "etes"), preserving distinguishing information.

Q2: Why not use a vocabulary of 1 million tokens?

A vocabulary of 1 million tokens would cause several problems:

1. **Enormous embedding matrix:** With embedding dimension 768 (like GPT-2), the embedding layer alone would have $1{,}000{,}000 \times 768 \approx 768$ million parameters—nearly as many as the entire GPT-2 model.
2. **Data sparsity:** Most tokens would appear very rarely in training data, leading to poorly trained embeddings for rare tokens.
3. **Overfitting:** The model memorizes rare token patterns instead of learning generalizable subword structure.
4. **Diminishing returns:** Beyond ∼100K tokens, additional vocabulary entries provide minimal compression benefit while significantly increasing model size.

The sweet spot (∼30K–100K) balances compression efficiency against embedding quality and model size.

Q3: UTF-8 byte tokenization of "hello" and a Chinese character

"hello": Each ASCII character is 1 byte in UTF-8, so "hello" = 5 bytes = **5 tokens**. This is identical to character-level tokenization for English.

A Chinese character: Most Chinese characters use **3 bytes** in UTF-8. So a single meaningful character becomes **3 tokens**. A 10-character Chinese sentence would become 30 tokens.

Implication: Raw UTF-8 byte tokenization creates **extremely long sequences** for non-English text, making it impractical. A 500-character Chinese text would become 1,500 byte tokens, dramatically increasing attention computation ($O(n^2)$). This is why we need BPE to merge frequent byte sequences into meaningful subword tokens.

Q4: Word-level tokenization and morphological waste

With word-level tokenization, "run", "running", "runner", and "runs" are four **completely independent** tokens with four separate embedding vectors. The model must learn from scratch that these words share the concept of "run"—it gets no help from the tokenizer.

With subword tokenization (BPE), these would be tokenized as:

- "run" → ["run"]

- `"running"` → `["run", "ning"]`
- `"runner"` → `["run", "ner"]`
- `"runs"` → `["run", "s"]`

All four words share the token `"run"`, so the model **automatically** transfers knowledge between them. The suffixes `"ning"`, `"ner"`, and `"s"` are also shared across many other words (e.g., "planning", "trainer", "cats"), further improving generalization.

Q5: Why merge the most frequent pair in BPE?

Merging the most frequent pair is a **greedy compression strategy**: at each step, we create the token that **reduces the total sequence length the most**. Frequent pairs appear many times, so merging them eliminates the most tokens from the corpus.

If we merged the **least** frequent pair instead:

- We would create tokens that appear only once or twice in the corpus
- The vocabulary would fill up with **rare, overly specific** tokens (e.g., merging "x" and "q" because they co-occur once)
- Common patterns (like "th", "in", "er") would never get merged because they would always be outranked by some rare pair
- The resulting tokenization would be **inefficient**: common words would still be split into many characters, while rare words get special tokens they do not deserve

The frequency-based approach ensures the vocabulary captures the most **useful** patterns first.

Q6: Token count comparison between GPT-2 and GPT-4 tokenizers

You would generally expect GPT-2's tokenizer to produce the **same number or more tokens** for "I love deep learning":

- GPT-4's tokenizer (`cl100k_base`) has **twice the vocabulary** of GPT-2's (`r50k_base`)
- A larger vocabulary means more common words and phrases are stored as single tokens
- Common English phrases like "deep learning" might be a single token in GPT-4 but two tokens in GPT-2

In this specific case, "I love deep learning" contains common English words, so both tokenizers may produce a similar count. But for longer or more varied text, GPT-4 consistently produces **fewer tokens**, which is one reason it is more cost-efficient.

Q7: Choosing a tokenizer for a multilingual + code model

Subword tokenization (byte-level BPE) is the clear choice. Here is why, language by language:

- **English:** Subword handles morphology naturally ("un" + "happy" + "ness"). Word-level would create OOV for rare terms; character-level would create unnecessarily long sequences.
- **Arabic:** The root-and-pattern morphology (e.g., k-t-b family) is naturally captured by subword units. Word-level would treat each inflected form as independent, missing the shared root structure. Character-level would work but create very long sequences.

- **Chinese:** Has no whitespace between words, so word-level tokenization requires a separate word segmentation step. Byte-level BPE handles this naturally—common characters and character combinations become single tokens through merges.
- **Code:** Programming languages have consistent syntactic patterns (keywords, operators, indentation). BPE learns these patterns and creates tokens for common constructs (`def`, `return`, `==`). Character-level would make code sequences extremely long; word-level would fail on variable names and nested syntax.

The byte-level variant is essential: it starts from 256 bytes, so it can handle **any encoding** without unknown tokens. This is exactly why GPT-4 uses byte-level BPE with a 100K vocabulary.